Contents

Prototype Design Pattern — From First Principles

---

1. The Problem (Before the Pattern)

Real-life analogy: The biology lab

Imagine a biology lab needs 1,000 identical cells for an experiment. Nobody sits down and manually assembles a cell from raw amino acids 1,000 times, following a blueprint step by step. Instead, they take **one working cell and let it divide (mitosis)** — each new cell is a **copy of an existing, already-working instance**, not a fresh build from scratch.

This is literally where the pattern's name comes from: a **prototype** is a fully-formed working example that you **copy**, rather than a blueprint you build from every time.

The concrete software problem

Say you have an object that is:

1. **Expensive to create** — e.g., it requires a network call, a database query, heavy computation, or parsing a large file to initialize.
2. **Complex to configure** — e.g., a `GameCharacter` with 40 stats, an AI-tuned `NeuralNetworkConfig`, or a fully-styled `Document` template.
3. **One of many near-identical variants** — e.g., spawning 500 "Goblin" enemies in a game that are 95% identical except position and a random health roll.

```
// The naive way — rebuild from scratch every single time
Goblin g1 = new Goblin();
g1.loadSprite("goblin.png");       // expensive - disk I/O
g1.loadSounds();                    // expensive - disk I/O
g1.setStats(10, 5, 2);              // repeated for every goblin
g1.setAIBehaviorTree(loadBehaviorTree("goblin_ai.json")); // expensive - parsing

Goblin g2 = new Goblin();
g2.loadSprite("goblin.png");        // reloading the SAME sprite, again
g2.loadSounds();                    // reloading the SAME sounds, again
g2.setStats(10, 5, 2);              // same numbers, again
g2.setAIBehaviorTree(loadBehaviorTree("goblin_ai.json")); // re-parsing the SAME file
```

Why this is genuinely bad engineering

Problem	Explanation
Wasted expensive work	You re-do the exact same costly initialization (disk I/O, DB calls, parsing) for every new object, even though 95% of the data is identical to one you already built.
Tight coupling to concrete classes	If a `Client` wants "another one of whatever this object is," it must call `new ConcreteClass()` directly — meaning the client must know and import the concrete class, breaking the "program to an interface" principle.
Hard to create objects when you don't know their exact type at compile time	If you receive an `Enemy` object from a plugin/config system and just want "another one just like this," you can't — you'd need a giant `if/else` or `switch` on its type to know which constructor to call.
Duplicated configuration logic scattered everywhere	Every place in code that needs a similarly-configured object re-writes the same setup steps, instead of reusing a known-good, already-configured instance.

In one sentence: the problem Prototype solves is — > *"How do I create new objects by copying an existing, fully-initialized instance — instead of re-running expensive/complex construction logic, and without the client needing to know the object's concrete class?"*

---

2. Standard (GoF) Definition

> **Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes. Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.** — *Design Patterns: Elements of Reusable Object-Oriented Software* (Gang of Four, 1994)

Key phrase to remember: **"copy existing objects without depending on their concrete classes."** The client just calls `.clone()` on whatever object it's holding — it never writes `new SomeSpecificClass()`.

---

3. How Prototype Solves It (in words)

Instead of the client constructing an object from scratch, Prototype:

1. Defines a common **`clone()`** method (via an interface or abstract class) that every "cloneable" type implements.
2. Each concrete class implements `clone()` to **return a new instance with the same field values as itself** — it knows its own internals best, so it's the right place to put copying logic.
3. The client simply calls `existingObject.clone()` — it never needs to know or import the concrete class, nor repeat the expensive setup.
4. Optionally, a **Prototype Registry** stores a catalog of ready-made prototypes so the client can request "give me a clone of type X" by name/key.

Real-life analogy: The photocopier

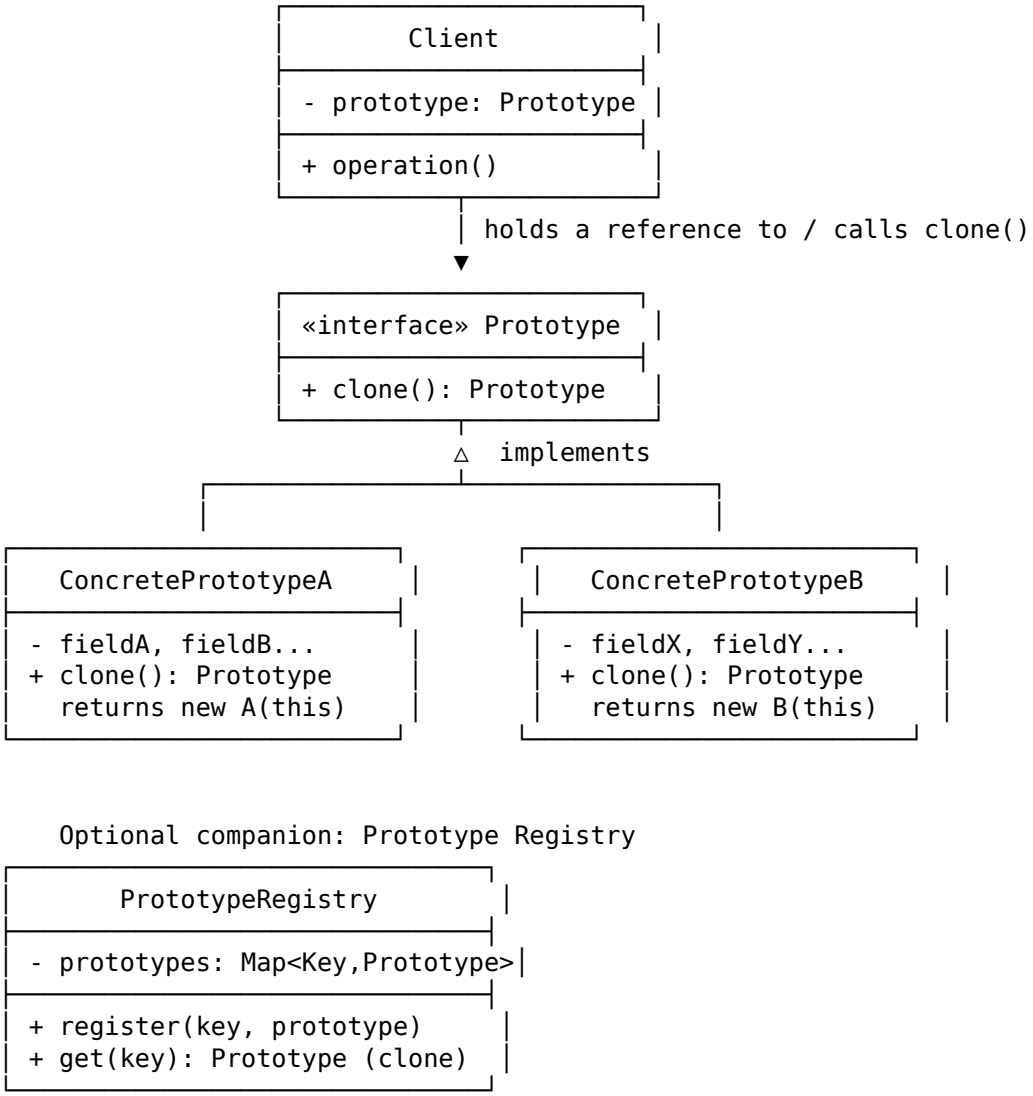
You need a signed contract for 50 clients. You don't re-type the entire 10-page contract from a blank page each time. You **photocopy the one you already have** and just fill in the blank name field on each copy. The **original signed contract is the prototype**; the photocopier's "copy" button is `clone()`.

But here's the subtlety that interviewers love: **what if the contract has a stapled-on appendix (a mutable object reference)?** - If you just photocopy the front page and physically staple the *same original appendix* to all 50 copies (**shallow copy**), then if someone scribbles on that one shared appendix, all 50 "copies" show the scribble. - If the photocopier also duplicates the appendix itself for each copy (**deep copy**), each of the 50 contracts has its own independent appendix — editing one never affects another.

This shallow-vs-deep distinction is the single most-asked follow-up question about this pattern.

---

4. UML Diagram



Relationships between the objects

Relationship	Type	Meaning
Client → Prototype (interface)	**Association**	Client holds a reference to the abstract `Prototype` type, never the concrete class. It only ever calls `.clone()`.
ConcretePrototype → Prototype	**Realization (implements)**	Each concrete type implements its own `clone()`, deciding internally how to duplicate itself (shallow or deep).
ConcretePrototype → ConcretePrototype (self)	**Self-referential creation**	Unlike Factory/Builder, the "recipe" for creating a new object *is* an existing object of the same type — `clone()` returns an object of the exact same runtime class as `this`.
PrototypeRegistry → Prototype	**Aggregation**	The registry holds a collection of pre-configured prototypes and hands out clones on request, keeping the client fully decoupled from concrete classes.

The core insight: In every other creational pattern (Factory Method, Builder, Abstract Factory), *the class itself* knows how to build a new instance from raw parameters. In Prototype, **an existing object instance is the source of truth** — new objects are stamped out from a working example, not assembled from a spec.

---

5. Code Example

5.1 Why Java's built-in `Cloneable` is considered broken (know this for interviews!)

```
class Goblin implements Cloneable {
    private int health;
    private List<String> lootTable; // mutable reference type

    public Goblin(int health, List<String> lootTable) {
        this.health = health;
        this.lootTable = lootTable;
    }

    @Override
    public Goblin clone() {
        try {
            return (Goblin) super.clone(); // Object.clone() - SHALLOW copy!
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(); // can never actually happen once Cloneable is implement
        }
    }
}
```

Problems with this approach (Joshua Bloch, *Effective Java*, Item 13): - `Cloneable` is a **marker interface with no methods** — it doesn't even declare `clone()`. You're implementing an interface that does nothing, and overriding a *protected* method from `Object` instead. This is a broken design contract. - `super.clone()` does a **shallow, field-by-field copy** — `lootTable` in the clone points to the **exact same List object** as the original. Modify the clone's loot table, and the original's loot table changes too. This is rarely what you want, and it's an easy-to-miss bug. - `clone()` **does not call any constructor** — so any invariant-checking or setup logic in your constructors is silently skipped. - Handling the checked `CloneNotSupportedException` is a needless annoyance. - `final` fields **cannot be reassigned** inside `clone()`, which makes deep-copying them awkward or impossible.

5.2 The idiomatic, interview-approved fix: Copy Constructor / Copy Factory

```
class Goblin {
    private int health;
    private List<String> lootTable;

    public Goblin(int health, List<String> lootTable) {
        this.health = health;
        this.lootTable = lootTable;
    }

    // Copy constructor - explicit, safe, and does a DEEP copy on purpose
    public Goblin(Goblin source) {
        this.health = source.health;
        this.lootTable = new ArrayList<>(source.lootTable); // new list, same contents -> deep c
    }

    public Goblin deepCopy() {
        return new Goblin(this);
    }
}

// Usage:
Goblin original = new Goblin(10, new ArrayList<>(List.of("Sword", "Gold")));
Goblin copy = original.deepCopy();
copy.lootTable.add("Shield");   // Only affects `copy`, NOT `original` - true independence
```

This is what most senior engineers actually recommend today: **skip `Cloneable` entirely**, use a copy constructor or a static `copyOf()` factory method instead. It runs through the real constructor, respects invariants, and makes the shallow/deep decision explicit and visible in code.

5.3 Prototype Interface (textbook GoF style, done properly)

```
interface Prototype {
    Prototype clone();
}

class Circle implements Prototype {
    private int radius;
    private String color;

    public Circle(int radius, String color) {
        this.radius = radius;
```

```
        this.color = color;
    }

    // Copy constructor used internally by clone()
    private Circle(Circle source) {
        this.radius = source.radius;
        this.color = source.color;
    }

    @Override
    public Circle clone() {
        return new Circle(this); // deep/shallow decided explicitly here
    }
}

// Client code - NEVER references `Circle` directly, only `Prototype`
class Client {
    public Prototype duplicate(Prototype p) {
        return p.clone(); // works for ANY Prototype implementation, unknown concrete type
    }
}
```

5.4 Prototype Registry (the "catalog" variant)

```
class ShapeRegistry {
    private Map<String, Prototype> prototypes = new HashMap<>();

    public void register(String key, Prototype prototype) {
        prototypes.put(key, prototype);
    }

    public Prototype get(String key) {
        return prototypes.get(key).clone(); // hand out a fresh clone, never the original
    }
}

// Usage:
ShapeRegistry registry = new ShapeRegistry();
registry.register("small-red-circle", new Circle(5, "red"));

Prototype c1 = registry.get("small-red-circle"); // independent clone #1
Prototype c2 = registry.get("small-red-circle"); // independent clone #2
```

---

6. Types of Prototype Pattern (FAANG Interview Depth)

Type 1 — Shallow Copy

Copies the object's own fields, but **any field that is a reference type (array, list, another object) is shared** between the original and the copy — both point to the *same* underlying object in memory.

Real-life analogy: Photocopying just the *cover page* of a binder, but physically handing over the **same original inner folder** of loose documents to both binders. If someone removes a page

from that shared folder, it disappears from *both* binders — because there was really only ever one folder.

When to use: When the referenced objects are immutable (e.g., `String`, immutable value objects), or when you deliberately *want* shared state (e.g., two objects intentionally sharing one big read-only cache).

Type 2 — Deep Copy

Recursively copies not just the object itself, but **every mutable object it references**, all the way down, so the copy is **fully independent** of the original.

Real-life analogy: A cloning lab that doesn't just copy a cell's outer membrane, but also duplicates every organelle inside it — mitochondria, nucleus, everything — so the clone can live and change completely independently of the original cell.

When to use: Whenever the object holds mutable state (lists, maps, nested objects) and you need true independence — e.g., an "undo" snapshot, or spawning game entities that must not share inventories.

Common implementation techniques: - Manual recursive copy constructors (most explicit, most recommended) - Serialization round-trip (serialize to bytes/JSON, then deserialize into a new object graph) — simple but has performance and versioning overhead - Deep-clone libraries (e.g., Kryo in Java) — convenient but adds a dependency

Type 3 — Prototype Registry / Prototype Manager

A centralized catalog of pre-configured prototype instances, keyed by name or type, from which clients request clones **without ever instantiating a concrete class themselves**.

Real-life analogy: A seed bank / nursery. You don't grow a new oak tree from a bare acorn every single time someone wants one in their garden — the nursery keeps a set of healthy, already-grown "mother plants" (prototypes) of each species, and simply propagates ("clones") a new one from cuttings whenever ordered. The customer just says "I want an oak" (a key), never how to grow one from scratch.

When to use: When you have many predefined "template" configurations (game enemy types, document templates, UI theme presets) and want the rest of the system to request them by name, fully decoupled from their concrete implementation classes.

⚠ Interview Trap — Don't confuse this with "Prototypal Inheritance"

JavaScript's `prototype` chain (`Object.create()`, `__proto__`) is a **language-level inheritance mechanism**, where objects delegate property lookups to a linked prototype object. It is **inspired by the same idea** (reuse an existing object instead of a class blueprint) but it is **not** the GoF Prototype *design pattern* — GoF Prototype is specifically about **cloning an object to produce a new, independent copy**, not about delegation/inheritance chains. Interviewers sometimes test whether you can tell these apart.

---

7. Use Cases (Real World)

Use Case	Where it appears
Game development	Spawning many enemies/NPCs/particles from a pre-configured "template" instance instead of rebuilding stats, sprites, and AI trees from scratch each time.
`Object.clone()` in Java	Language-level support for the pattern, though widely discouraged in favor of copy constructors (see Section 5.1).
Document/spreadsheet "Duplicate" or "Make a copy" features	Google Docs, Photoshop layers, Figma components — the new file/layer/component is a clone of an existing one, not built from a blank template.
Undo/Redo systems	Taking a deep-copy "snapshot" of application state before a mutation, so you can restore it later without re-running whatever built the original state.
Expensive database-backed objects / caching layers	Cloning an already-fetched, fully-hydrated object instead of re-querying the database for a near-identical object.
Configuration/template objects	Cloning a base `HttpRequestTemplate` or `ReportConfig` and tweaking only a couple of fields per use, instead of rebuilding the whole config each time.
Spring Framework's `prototype` bean scope	Related in spirit (a new instance per request) — though this is Spring instantiating a new bean per request rather than literally cloning an existing object, it borrows the same name and reuse philosophy.

---

8. Prototype vs Other Creational Patterns (Quick Interview Differentiator)

Pattern	Difference from Prototype
Factory Method	Creates a new object via **subclass-specific construction logic** — the class itself decides how to build a fresh instance. Prototype instead **copies an existing, already-built instance** — no "build from parameters" logic is invoked at all.
Builder	Assembles a complex object **step by step from raw parts**, typically producing a fresh object with no prior instance. Prototype **skips construction entirely** by duplicating something that already exists.
Abstract Factory	Produces **families of related objects** through factory interfaces. Prototype produces new instances of **a single object** by copying, and doesn't inherently deal with families.

Pattern	Difference from Prototype
Singleton	Ensures **only one instance ever exists**. Prototype does the opposite — it exists specifically to **produce many independent instances** cheaply from one template.

---

9. Memory Map (For Quick Recall Before an Interview)

```
PROTOTYPE PATTERN — MEMORY MAP
│
├── PROBLEM
│   └── Object creation is expensive/complex, OR client shouldn't need to know
│       the concrete class just to get "another one like this"
│
├── DEFINITION (say this verbatim)
│   └── "Copy existing objects without depending on their concrete classes"
│
├── KEY PLAYERS
│   ├── Prototype (interface)     → declares clone()
│   ├── ConcretePrototype          → implements clone(), knows its own copy logic
│   ├── Client                    → holds Prototype reference, calls .clone() only
│   └── PrototypeRegistry (opt.)  → catalog of named prototypes, hands out clones
│
├── RELATIONSHIPS
│   ├── Client --uses--> Prototype (interface, never concrete class)
│   ├── ConcretePrototype --implements--> Prototype
│   └── clone() --returns--> new instance of the SAME concrete class as `this`
│
├── 3 TYPES (mnemonic: "Shallow Deep Registry")
│   ├── 1. Shallow Copy  -> top-level fields only, shares refs  [photocopy cover, share folder]
│   ├── 2. Deep Copy    -> recursively independent copy      [cloning lab copies organelles too]  mo
│   └── 3. Prototype Registry -> catalog of named templates      [seed bank / nursery]
│
├── ⚠ JAVA-SPECIFIC GOTCHA
│   └── Avoid `Cloneable`/`Object.clone()` -> broken marker interface, shallow by
│       default, skips constructors. PREFER: copy constructor / static copyOf()
│
├── ⚠ DON'T CONFUSE WITH
│   └── JavaScript "prototypal inheritance" (delegation chain) — related idea,
│       NOT the same as the GoF Prototype cloning pattern
│
├── REAL WORLD
│   └── Game entity spawning, "Duplicate" in Docs/Figma/Photoshop, undo/redo
│       snapshots, cloning expensive DB-hydrated objects
│
└── DIFFERENTIATE FROM
    ├── Factory Method -> builds fresh instance via class logic
    ├── Builder        -> assembles fresh instance step-by-step
    ├── Abstract Factory-> families of related objects
    └── Singleton      -> opposite goal: only ONE instance ever
```

One-line summary to say out loud in an interview:

> *"Prototype creates new objects by cloning an existing, fully-initialized instance rather than building from scratch — avoiding expensive re-construction and decoupling the client from concrete classes — with the critical design decision being whether that clone is shallow or deep."*